

Unit 2: HTML (7 hrs.)

Compiled By

Lecturer, Sunil Bahadur Bist

NAST, Dhangadhi

Contents

2.1 Introduction

- 2.1.1 History and evolution of HTML

- 2.1.2 Concept and types of tag

- 2.1.3 HTML document structure

- 2.1.4 HTML elements and attributes

- 2.1.5 Basic text formatting tags

2.2 Lists, Links

- 2.2.1 Unordered, ordered and description lists

- 2.2.2 Creating hyperlinks, types of links

2.3 Adding images to a web page

Contents

2.4 Table and Forms

- 2.4.1 html table elements and attributes, merging row and columns

- 2.4.2 Common form elements like text fields, text areas, selects, buttons, radio button.

- 2.4.3 input date, time

- 2.4.4 form methods

2.5 Semantic HTML

- 2.5.1 Header, footer, nav, article, section

2.6 Audio and video elements

2.7 div, span

What is HTML?

- HTML stands for **HyperText Markup Language**.
- It is the standard language used to create and design web pages.
- HTML tells the web browser:
 - what content to display,
 - how the content is structured,
 - and how different elements like text, images, links, tables, and forms appear on a webpage.
- HTML is called:
- **HyperText** because it allows linking between web pages.
- **Markup Language** because it uses tags to structure content.

Features of HTML

- Simple and easy to learn
- Platform independent
- Used to create web pages
- Supports images, audio, video, tables, and forms
- Works with CSS and JavaScript

Why to Learn HTML?

- Learning HTML is valuable because it's the **foundation of the web**. Here's why it's worth your time:

1. It's the building block of websites

HTML (HyperText Markup Language) structures every webpage you see: headings, paragraphs, images, links, forms, everything.

2. Easy entry into tech

HTML is one of the simplest coding languages to start with. You don't need prior programming experience.

3. Essential for web development

If you want to become a web developer, HTML is the first step along with CSS (design) and JavaScript (interactivity).

Why to Learn HTML?

4. Helps you customize content

Whether it's editing a blog, email template, or website, HTML lets you control how content appears.

5. Works with other technologies

HTML is used alongside frameworks, CMS platforms (like WordPress), and even apps—so it opens many doors.

6. In-demand skill

Even basic HTML knowledge is useful in jobs like digital marketing, content creation, design, and tech roles.

7. You can build real things quickly

With just HTML, you can create a simple webpage in minutes and see results instantly.

Applications of HTML

- HTML has many practical applications across the web and software world. Here are the main uses of HTML:
- **Creating Web Pages**
HTML is primarily used to build and structure webpages with headings, text, images, links, tables, and forms.
- **Web Applications**
Modern web apps use HTML together with CSS and JavaScript to create interactive platforms like social media sites, online editors, and dashboards.
- **Website Navigation**
HTML creates menus, hyperlinks, and navigation systems that connect webpages together.

Applications of HTML

- **Embedding Multimedia**

It allows developers to add audio, video, animations, maps, and graphics to websites.

- **Online Forms**

HTML forms collect user information such as login details, feedback, registrations, and search queries.

- **Email Templates**

Many marketing and business emails are designed using HTML for better formatting and visuals.

- **Documentation Pages**

HTML is widely used for creating online manuals, tutorials, technical documentation, and e-books.

Applications of HTML

- **Mobile App Interfaces**
Some mobile apps use HTML with technologies like hybrid app frameworks to build cross-platform interfaces.
- **Game Development**
Browser-based games often use HTML together with JavaScript and canvas elements.
- **Data Presentation**
HTML tables and layouts help display reports, schedules, statistics, and structured information clearly.
- **SEO (Search Engine Optimization)**
Proper HTML structure helps search engines understand and rank webpages better.
- **Integration with Other Technologies**
HTML works with CSS, JavaScript, databases, APIs, and frameworks to create complete websites and applications.

Limitations of HTML

- **No Logic or Functionality:** HTML cannot handle complex logic, interactivity, or dynamic content on its own. It requires JavaScript for such tasks.
- **SEO Limitations:** While HTML provides structure, it's not enough by itself for search engine optimization (SEO). Proper metadata and content structuring, as well as external SEO practices, are necessary.
- **Limited Styles:** While HTML can handle basic styles via the style attribute, it is typically complemented by CSS for complex styling and layout.

History and Evolution of HTML

- HTML was developed by Tim Berners-Lee in 1991.
- **Evolution of HTML**

Version	Release Date	Description
HTML 1.0	1993	Basic web pages
HTML 2.0	1995	Standardized version
HTML 3.2	1997	Added tables and scripting
HTML 4.01	1999	Improved styling support
XHTML	2000	Stricter syntax rules
HTML5	2014	Modern multimedia and semantic web features

Concept and Types of Tag

- HTML uses **tags** enclosed in angle brackets <>

Types of Tags

1. Container/Paired Tags

- Have opening and closing tags.

For example, <p>This is paragraph</p>

2. Empty/Unpaired Tags

- Do not need closing tags.

For example,
, <hr>,

HTML Document Structure

- Basic structure of an HTML document:

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <title>My Web Page</title>
```

```
</head>
```

```
<body>
```

```
  <h1>Welcome</h1>
```

```
  <p>Hello World!</p>
```

```
</body>
```

```
</html>
```

Main Parts

- `<!DOCTYPE html>` : Defines HTML5 document
- `<html>` : Root element
- `<head>` : Metadata section
- `<title>` : Browser tab title
- `<body>` : Visible webpage content
- `<h1>` : Bigger heading
- `<p>` : paragraph

HTML Elements and Attributes

Element

- Combination of opening tag, content, and closing tag.

`<p>This is a paragraph</p>`

Attribute

- Provides extra information to elements.

Syntax:

`<tag attribute="value">`

for example, `<a href="https://nast.edu.np">NAST College</a>`

HTML Elements and Attributes

Common attributes:

- id
- class
- href
- src
- style

Basic Text Formatting Tags

- `<b>`: Bold text
- `<i>`: Italic text
- `<u>`: Underline text
- `<strong>`: Important text
- `<em>`: Emphasized text
- `<mark>`: Highlight text
- `<small>`: Small text
- `<sup>`: Superscript
- `<sub>`: Subscript

Basic Text Formatting Tags

Example:

- `<b>Bold</b>`
- `<i>Italic</i>`
- `<u>Underline</u>`

Lists and Links

- An HTML List allows you to organize data on web pages into an ordered or unordered format to make the information easier to read and visually appealing.
- HTML Lists are very helpful for creating structured, accessible content in web development.

Types of HTML Lists

- There are three main types of lists in HTML:
- **Unordered Lists ():** These lists are used for items that do not need to be in any specific order. The list items are typically marked with bullets.
- **Ordered Lists ():** These lists are used when the order of the items is important. Each item in an ordered list is typically marked with numbers or letters.
- **Description Lists (<dl>):** These lists are used to contain terms and their corresponding descriptions.

HTML List Tags

Tag	Description
<u></u>	Defines an unordered list.
<u></u>	Defines an ordered list.
<u></u>	Defines a list item.
<u><dl></u>	Defines a description list.
<u><dt></u>	Defines a term in a description list.
<u><dd></u>	Details the term in a description list.

HTML List Types

Attribute: This tag contains two attributes which are listed below:

- [compact](#): It will render the list smaller.
- [type](#): It specifies which kind of marker is used in the list.

Values of type

- disc: solid round bullets (default)
- circle: hollow round bullets
- square: square bullets

1. Unordered List

- Uses bullets.

```
<ul>  
    <li>Apple</li>  
    <li>Mango</li>  
</ul>
```

```
<ul type="square">  
    <li>HTML</li>  
    <li>CSS</li>  
    <li>JavaScript</li>  
</ul>
```

HTML List Types

Attributes:

- [compact](#): It defines the list should be compacted (compact attribute is not supported in HTML5. Use CSS instead.).
- [reversed](#): It defines that the order will be descending.
- [start](#): It defines from which number or alphabet the order will start.
- [type](#): It defines which type(1, A, a, I, and i) of the order you want in your list of numeric, alphabetic, or roman numbers.

2. Ordered List

- Uses numbering.

```
<ol>  
    <li>HTML</li>  
    <li>CSS</li>  
</ol>
```

```
<ol style="line-height: 0.8;">  
    <li>HTML</li>  
    <li>CSS</li>  
    <li>JavaScript</li>  
</ol>
```

HTML List Types

3. Description List

- Used for terms and descriptions.

`<dl>`

`<dt>HTML</dt>`

`<dd>Markup language</dd>`

`</dl>`

HTML Links (Hyperlinks)

- **HTML Links**, also known as **hyperlinks**, are defined by the **<a>** tag in HTML, which stands for “anchor”.
- These links are essential for navigating between web pages and directing users to different sites, documents, or sections within the same page.
- The basic attributes of the **<a>** tag include **href**, **title**, and **target**, among others.

Hyperlink Syntax

```
<a href="URL">Link Text</a>
```

Types of Links

1. External Link

`<a href="https://google.com">Google</a>`

2. Internal Link

`<a href="about.html">About</a>`

3. Email Link

`<a href="mailto:test@gmail.com">Send Email</a>`

4. Bookmark Link

`<a href="#section1">Go to Section</a>`

Hyperlinks Behaviors & Target Attribute

- By default, links will appear as follows in all browsers:
- An **unvisited link** is underlined and blue.
- A **visited link** is underlined and purple.
- An **active link** is underlined and red.

HTML Links – Target Attribute:

- The target attribute in the <a> tag specifies where to open the linked document.
- It controls whether the link opens in the same window, a new window, or a specific frame.

HTML Links – Target Attribute

Attribute	Description
_blank	Opens the linked document in a new window or tab.
_self	Opens the linked document in the same frame or window as the link. (Default behavior)
_parent	Opens the linked document in the parent frame.
_top	Opens the linked document in the full body of the window.
framename	Opens the linked document in a specified frame. The frame's name is specified in the attribute.

Linking Different HTML Elements

Element to Interlink	Specific Code
Image as link	<code></code>
Email as Link	<code>Send Email</code>
Phone Number	<code>Call Now</code>
Button	<code> <button>Visit Website</button> </code>
Link to Download File	<code>Download File</code>
Link Title	<code>Website Link Text</code>

Linking Part of Section

```
<!DOCTYPE html>
<html lang="en">
<head>
  <title>Linking to Anchors</title>
</head>
<body>
  <nav>
    <ul>
      <li><a href="#section1">Go to Section 1</a></li>
      <li><a href="#section2">Go to Section 2</a></li>
      <li><a href="#section3">Go to Section 3</a></li>
    </ul>
  </nav>
```

Linking Part of Section

```
<h2 id="section1">Section 1</h2>  
<p>This is the content of section 1.</p>
```

```
<h2 id="section2">Section 2</h2>  
<p>This is the content of section 2.</p>
```

```
<h2 id="section3">Section 3</h2>  
<p>This is the content of section 3.</p>  
</body>  
</html>
```

Adding Images to a Web Page

- The **HTML tag** is used to embed an image in web pages by linking them.
- It creates a placeholder for the image, defined by attributes like src, width, height, and alt, and does not require a closing tag.
- There are **two ways** to insert the images into a webpage:
 - By providing a full path or address (URL) to access an internet file.
 - By providing the file path relative to the location of the current web page file.

Adding Images to a Web Page

Image Syntax

```

```

Important Attributes

- src: Image path
- Alt: Alternative text
- width and height: Image size

Example, ``

Table and Forms

- HTML Tables allow you to arrange data into rows and columns on a web page, making it easy to display information like schedules, statistics, or other structured data in a clear format.
- An HTML table is created using the `<table>` tag.
- Basic HTML Table Structure
- An HTML table is created using the `<table>` tag.
- Inside the table, we use:
 - `<tr>`: Represents a row within the table.
 - `<th>`: Represents the header cells in the table.
 - `<td>`: Represents a regular data cell within the table.

Table and Forms

- Each `<tr>` represents a row, and within each row, `<th>` or `<td>` tags represent the cells in that row, which can contain text, images, lists, or even another table.

Table Example

```
<table border="1">
  <tr>
    <th>Name</th>
    <th>Age</th>
  </tr>
  <tr>
    <td>Ram</td>
    <td>20</td>
  </tr>
</table>
```

Tags used in HTML Tables

Tags	Descriptions
<u><table></u>	Defines the structure for organizing data in rows and columns within a web page.
<u><tr></u>	Represents a row within an HTML table containing individual cells.
<u><th></u>	Shows a table header cell that typically holds titles or headings.
<u><td></u>	Represents a standard data cell, holding content or data.
<u><caption></u>	Provides a title or description for the entire table.
<u><thead></u>	Defines the header section of a table, often containing column labels.
<u><tbody></u>	Represents the main content area of a table, separating it from the header or footer.
<u><tfoot></u>	Specifies the footer section of a table, typically holding summaries or totals.

Table Tags

Merging Rows and Columns

Column Merge (colspan)

```
<td colspan="2">Merged Column</td>
```

Row Merge (rowspan)

```
<td rowspan="2">Merged Row</td>
```

Classwork

Employee Information Table

ID	Name	Department	Salary
101	Alice	Engineering	5000
	Bob	Engineering	5200
102	Charlie (HR Manager)		6000
103	David	Finance	5500
Total Employees			4

HTML Code

```
<!DOCTYPE html>
<html lang="en">
<head>
<title>Table Example</title>
</head>
<body>

<!-- FULL TABLE EXAMPLE -->
<table
  border="1"
  cellpadding="5"
  cellspacing="0"
```

```
width="100%"
  align="center"
  bgcolor="#ffffff"
>

<!-- TABLE CAPTION -->
  <caption>Employee Information
Table</caption>

<!-- TABLE HEADER -->
<thead>
<tr>
```

```
<th>ID</th>
<th>Name</th>
<th>Department</th>
<th>Salary</th>
</tr>
</thead>

<!-- TABLE BODY -->
<tbody>
<tr>
  <td rowspan="2">101</td>
```

HTML Code...

```
<td>Alice</td>
  <td>Engineering</td>
  <td>5000</td>
</tr>

<tr>
  <td>Bob</td>
  <td>Engineering</td>
  <td>5200</td>
</tr>

<tr>
  <td>102</td>
```

```
  <td colspan="2">Charlie
(HR Manager)</td>
  <td>6000</td>
</tr>

<tr>
  <td>103</td>
  <td>David</td>
  <td>Finance</td>
  <td>5500</td>
</tr>
</tbody>

<!-- TABLE FOOTER -->
```

```
<tfoot>
  <tr>
    <td colspan="3">Total
Employees</td>
    <td>4</td>
  </tr>
</tfoot>

</table>

</body>
</html>
```

HTML Forms

- HTML Forms use the `<form>` tag to collect user input through various interactive controls.
- These controls range from text fields, numeric inputs, and email fields to password fields, checkboxes, radio buttons, and submit buttons.
- **Syntax:**
 - `<form>`
 - `<!--form elements-->`
 - `</form>`

Form Elements

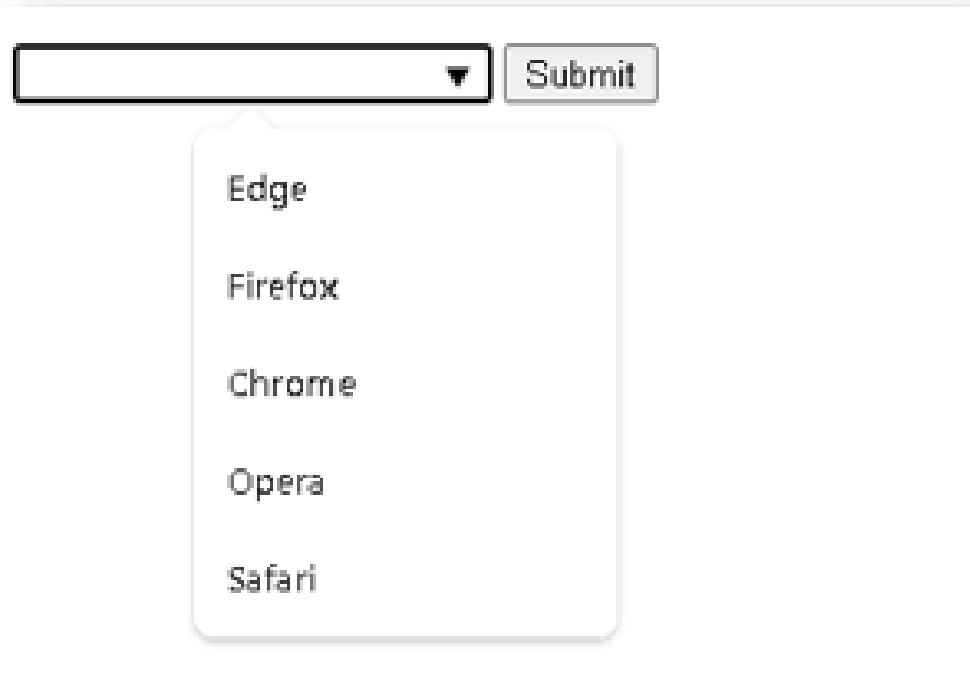
Elements	Descriptions
<u><label></u>	It defines labels for <form> elements.
<u><input></u>	It is used to get input data from various types such as text, password, email, etc by changing its type.
<u><button></u>	It defines a clickable button to control other elements or execute a functionality.
<u><select></u>	It is used to create a drop-down list.
<u><textarea></u>	It is used to get input long text content.
<u><fieldset></u>	It is used to draw a box around other form elements and group the related data.
<u><legend></u>	It defines a caption for fieldset elements
<u><datalist></u>	It is used to specify pre-defined list options for input controls.
<u><output></u>	It displays the output of performed calculations.
<u><option></u>	It is used to define options in a drop-down list.
<u><optgroup></u>	It is used to define group-related options in a drop-down list.

Commonly Used Input Types

Input Type	Description
<u><input type="text"></u>	Defines a one-line text input field
<u><input type="password"></u>	Defines a password field
<u><input type="submit"></u>	Defines a submit button
<u><input type="reset"></u>	Defines a reset button
<u><input type="radio"></u>	Defines a radio button
<u><input type="email"></u>	Validates that the input is a valid email address.
<u><input type="number"></u>	Allows the user to enter a number. You can specify min, max, and step attributes for range.
<u><input type="checkbox"></u>	Used for checkboxes where the user can select multiple options.
<u><input type="date"></u>	Allows the user to select a date from a calendar.
<u><input type="time"></u>	Allows the user to select a time.
<u><input type="file"></u>	Allows the user to select a file to upload.

Input type & data list example

```
<form action="action_page.py">  
  <input list="browsers" name="browser">  
  <datalist id="browsers">  
    <option value="Edge">  
    <option value="Firefox">  
    <option value="Chrome">  
    <option value="Opera">  
    <option value="Safari">  
  </datalist>  
  <input type="submit">  
</form>
```



The image shows a web form with a dropdown menu and a submit button. The dropdown menu is open, displaying a list of browser names: Edge, Firefox, Chrome, Opera, and Safari. The submit button is labeled "Submit".

Classwork

Username:

Password:

Submit

Name

Email

Check

Age

Country

Password

Resume

Choose File

No file chosen

Hobbies

☒

Cricket

☐

Football

Gender

☐

Female

☐

Male

City



Address

Submit

Reset

Form Methods

- GET and POST are two common HTTP methods used in web forms and APIs.

GET Method

- Used to **request/fetch data** from a server.

Characteristics

- Data is sent in the **URL**
- Visible in browser address bar
- Less secure
- Limited data length
- Can be bookmarked
- Faster and cacheable

GET Example

```
<form method="GET" action="/search">  
  <input type="text" name="item">  
  <button type="submit">Search</button>  
</form>
```

URL after submission:
/search?item=laptop

laptop	Search
--------	--------

Form Methods

POST Method

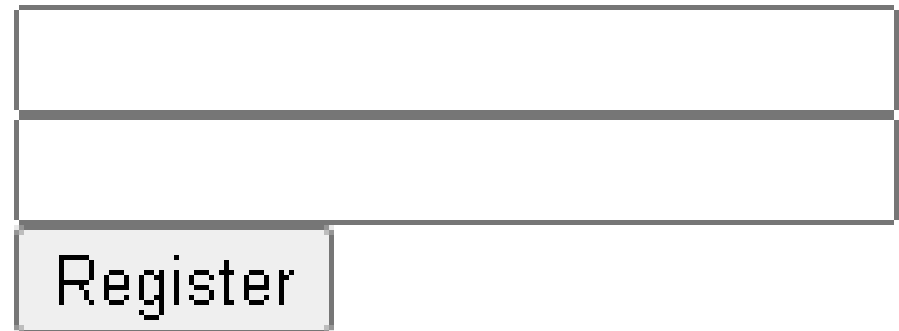
- Used to **send data** to the server.

Characteristics

- Data is sent in the **request body**
- Not visible in URL
- Can send large data/files
- More secure than GET
- Not usually cached

POST Example

```
<form method="POST" action="/register">  
  <input type="text" name="username"> <br/>  
  <input type="password" name="password"><br/>  
  <button type="submit">Register</button>  
</form>
```



A visual representation of the HTML form. It consists of two stacked rectangular input fields. The top field is for the username, and the bottom field is for the password. Below these fields is a button labeled "Register".

Main Differences

GET	POST
Retrieves data	Sends data
Data in URL	Data in body
Less secure	More secure
Limited size	Large size allowed
Bookmark possible	Bookmark not possible
Used for search/read	Used for login/signup/upload

Assignment

- Write down the HTML code for the output below:

Text	Password	Email Address
Number	Search	URL Address
-----	mm/dd/yyyy	Week --, ----
☒ Radio Button	<input type="range" value="50"/>	☐ Checkbox
Submit	Reset	Button

HTML5 Semantics

- HTML5 introduced a range of **semantic elements** that clearly describe their purpose in human and machine-readable language.
- Unlike non-semantic elements, which provide no information about their content, semantic elements clearly define their content.
- For instance, **<form>**, **<table>**, and **<article>** tags clearly define the content and purpose, to the browser.

Why Use Semantic HTML Tags?

- **Accessibility**

- Semantic elements make web pages more accessible.
- Screen readers and other assistive technologies can interpret the structure and navigate the content more efficiently.

- **SEO**

- Better structured data leads to better SEO.
- Search engines prioritize well-structured content that uses semantic elements correctly, as it's easier to index.

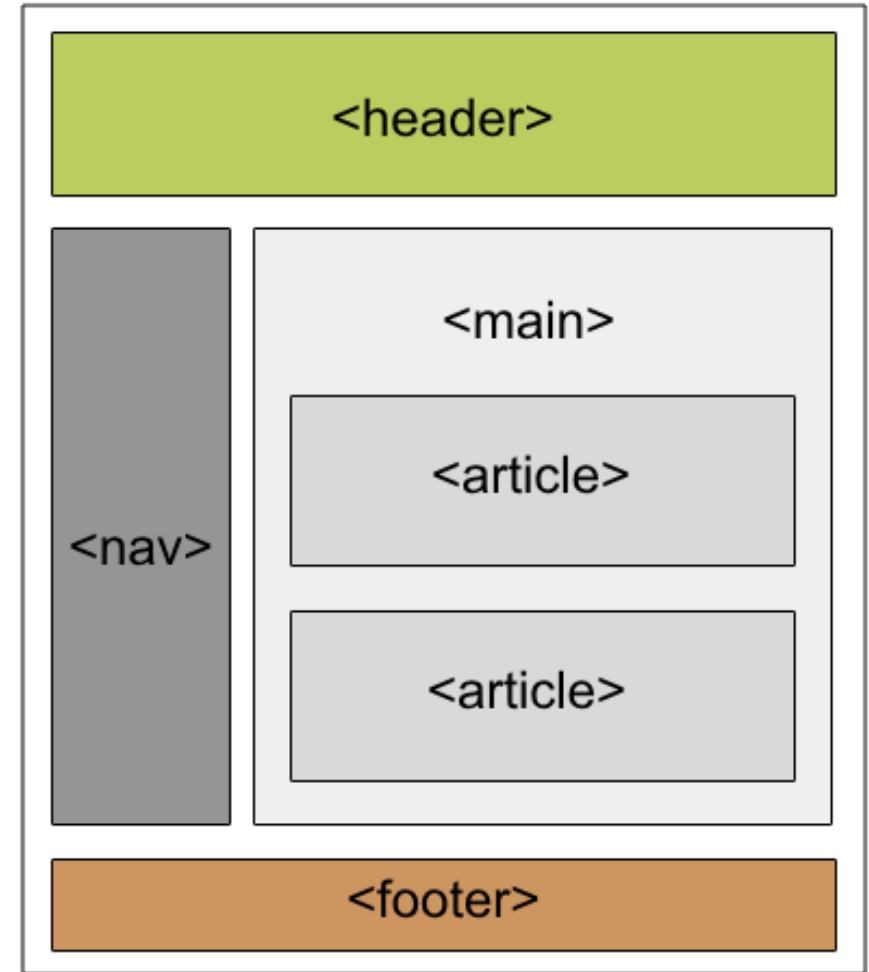
- **Maintainability**

- Semantic HTML helps create a logically structured document, which is easier to read and maintain.

Semantic Elements

- Here are some of the fundamental HTML5 semantic elements that you should use to structure your web content:

- `<header>`: Header section
- `<footer>`: Footer section
- `<nav>`: Navigation links
- `<article>`: Independent content
- `<section>`: Content section



HTML <header> Tag

- The **<header> tag** is a semantic HTML element that is used to define the introductory or navigational content of a webpage or a section.
- Typically, a header contains elements like:
 - *The website or page title*
 - *Logo or branding*
 - *Navigation menus*
 - *Search bar*
 - *Any introductory information relevant to the page or section*
- **Syntax:** <header> ...</header>
- **Note:** Header tag cannot be placed within a [<footer>](#), [<address>](#), or another [<header>](#) element.

HTML <footer> Tag

- The **<footer>** tag in HTML is used to define the footer section of an HTML document.
- The footer section typically contains information such as contact information, sitemap, back-to-top links, related documents, copyright, etc.
- The footer tag is a semantic tag included in HTML (in 2014) along with other tags like article, nav, header, etc.
- It is not mandatory, but adds to clear structure to the document and useful for SEO.

Syntax: <footer> </footer>

HTML <nav> Tag

- The <nav> tag in HTML is used to define navigation sections on a webpage.
- It typically contains links to important sections of the website, such as menus, tables of contents, or indexes, and is often structured within unordered lists () or as standalone links.

How to Use the <nav> Tag

- Links within the <nav> tag can either be standalone or structured within an unordered list () for better organization.
- While it's common to use lists, it's not a strict requirement.
- **Syntax:** <nav> <!-- Your navigation links here --> </nav>

HTML <nav> Example

<nav>

Home |

Interview |

Languages |

Data Structure |

Algorithm

</nav>

HTML <article> Tag

- The **HTML <article> tag** defines a self-contained, independent piece of content like a blog post, news article, or comment.
- It is designed for content that can be independently distributed, shared, or reused, providing semantic meaning to the content.
- This tag is introduced in HTML5. For example,

<article>

<h3>My First Blog Post**</h3>**

<p>

This is a brief introduction to my blog.

I will share my thoughts on various topics.

</p>

</article>

HTML <section> Tag

- **The Section tag** defines the section of documents such as chapters, headers, footers, or any other sections.
- The section tag divides the content into sections and subsections.
- The section tag is used when requirements of two headers or footers or any other section of documents are needed.
- Section tag grouped the generic block of related contents.
- The main advantage of the section tag is, it is a semantic element, that describes its meaning to both the browser and the developer.
- Syntax: <section> Section Contents </section>
- **Note:** Section tag is used to distribute the content i.e., it distributes the sections and subsections.

HTML <section> Tag Example

```
<section>
```

```
  <h1>
```

```
    Section 1
```

```
  </h1>
```

```
  <p>
```

```
    Content of section 1
```

```
  </p>
```

```
</section>
```

```
<section>
```

```
  <h1>
```

```
    Section 2
```

```
  </h1>
```

```
  <p>
```

```
    Content of section 2
```

```
  </p>
```

```
</section>
```

```
<section>
```

```
  <h1>
```

```
    Section 3
```

```
  </h1>
```

```
  <p>
```

```
    Content of section 3
```

```
  </p>
```

```
</section>
```

Semantic HTML Example

```
<header>  
  <h1>My Website</h1>  
</header>
```

```
<nav>  
  <a href="#">Home</a>  
</nav>
```

```
<section>
```

```
<article>  
  <h2>News</h2>  
  <p>Latest update...</p>  
</article>  
</section>
```

```
<footer>  
  Copyright 2026  
</footer>
```

Audio and Video Elements

- In HTML, **audio** and **video** elements are used to embed media directly into web pages without needing external plugins.

<audio> Tag

- We use the <audio> tag to embed audio in HTML.
- Before HTML5, audio cannot be added to web pages in the Internet Explorer era.
- To play audio, we used web plugins like Flash.
- After the release of HTML5, it is possible.
- This tag supports Chrome, Firefox, Safari, Opera, and Edge in three audio formats – MP3, WAV, OGG.
- Only Safari browser doesn't support OGG audio format.

Audio and Video Elements

- **Syntax:**

```
<audio>
```

```
  <source src="file_name" type="audio_file_type">
```

```
</audio>
```

For example,

```
<audio controls>
```

```
  <source src="song.mp3" type="audio/mp3">
```

```
</audio>
```

Attributes of <audio> tag

Attribute	Value	Description
autoplay	autoplay	When the page is loaded. It specifies to play audio as soon as possible.
controls	controls	It displays audio control.
loop	loop	It will start the audio again when it is finished.
muted	muted	When the page is loaded audio will be automatically muted.
preload	auto metadata none	It specifies how the author thinks the audio will be loaded when the page is ready.
src	URL	It specifies the URL of the audio file.

<video> Tag

- To embed video in HTML, we use the <video> tag.
- It contains one or more video sources at a time using <source> tag.
- It supports MP4, WebM, and Ogg in all modern browsers.
- Only Ogg video format doesn't support in Safari browser.

- **Syntax:**

```
<video>
```

```
  <source src="file_name" type="video_file_type">
```

```
</video>
```

Attributes of <video> tag

Attribute	Value	Description
autoplay	autoplay	When the page is loaded. It specifies to play video as soon as possible.
controls	controls	It displays video control such as play, pause, and stop.
loop	loop	It will start the video again when it is finished.
muted	muted	When the page is loaded video will be automatically muted.
poster	URL	It specifies an image will be shown until video play.
preload	auto metadata none	It specifies how the author thinks the video will be loaded when the page is ready.
Src	URL	It specifies the URL of the audio file.
width	pixels	It specifies the width of the video area. The default value of width is 'auto'.
height	pixels	It specifies the height of the video area. The default value of height is 'auto'.

Video Tag Example

```
<video width="400" controls>  
  <source src="movie.mp4" type="video/mp4">  
</video>
```

OR

```
<video controls preload="auto" autoplay muted loop width="600"  
height="400" src="vds.mp4"></video>
```

div and span

- In HTML, `<div>` and `<span>` are both generic containers used for grouping and styling content, but they have different display behaviors.
- `<div>` is a block-level element, meaning it starts on a new line and takes up the full width available.
- `<span>` is an inline element, fitting within the flow of text and only taking up as much space as the content within it requires.
- Here's a more detailed breakdown:

<div> (Block Element)

- **Purpose:**

- Used to create sections or divisions in a document, grouping other elements together and often used for layout purposes.

- **Display:**

- display: block by default, meaning it starts on a new line and takes up the full available width.

- **Example:**

- You might use a <div> to structure a webpage into a header, main content area, and footer.

- Code

```
<div>  
  <h1>My Web Page</h1>  
  <p>This is some content. </p>  
</div>
```

 (Inline Element)

- **Purpose:**

- Used to group inline text elements and apply styling or scripts to specific parts of the text.

- **Display:**

- display: inline by default, meaning it flows within the text and only occupies the space needed by its content.

- **Example:**

- You might use a to highlight specific words in a paragraph or change their color.

Code:

```
<p>This is some <span style="color: blue;">text</span> in a paragraph. </p>
```

Key Differences

Feature	<code><div></code>	<code></code>
Display	Block (takes up full width)	Inline (flows with text)
Usage	Structuring sections and layout	Styling inline elements or text
Newline	Starts on a new line	Does not start on a new line
Content	Can contain other block or inline elements	Can only contain inline elements

Example

```
1 div{  
2   display: block;  
3 }
```



```
1 div{  
2   display: inline;  
3 }
```



Next Chapter....